CSE752: Deep Neural Networks

Contents

Introduction to Neural Network **2**

Neurons and Logistic Regression **5**

Introduction to Neural Network

Computation of a Single Neuron: $f(w^\top x + b)$

A neuron computes $f(w^\top x + b)$, because the input is usually a vector.

- **Single input (scalar):** $a = f(wx + b)$
- **Multiple inputs (vector $x \in \mathbb{R}^d$):** $z = w^\top x + b$, then $a = f(z)$

The neuron does two things: an affine step (weighted sum plus bias) that gives the pre-activation z, then a nonlinearity f applied to that scalar. The nonlinearity is the only place a curve can enter. If f were the identity, stacking layers would collapse into one affine map.

For a whole layer, the same idea is written with a matrix: $z = Wa^{(l-1)} + b$, $a = f(z)$, with f applied elementwise.

Number of Inputs to a Neuron

A neuron can take multiple values as input. A single input is just the special case $d = 1$.

With d inputs, the neuron has **one weight per input** but only **one bias**: $x \in \mathbb{R}^d$, $w \in \mathbb{R}^d$, $b \in \mathbb{R}$. It multiplies each input by its own weight, sums the results, adds the bias, and applies f:

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b = w^\top x + b, \qquad a = f(z)$$

Whatever the number of inputs, the output is always **one scalar**.

For example, in a $784 \to 128 \to 64 \to 10$ network, each neuron in the first hidden layer takes all 784 inputs, so it has 784 weights and 1 bias. This is why a layer's parameter count is $n_{\text{out}} \cdot n_{\text{in}} + n_{\text{out}}$: n_{out} neurons, each with n_{in} weights and one bias.

What Is a Layer?

A layer is a group of neurons that all receive the same input vector and compute in parallel, each producing one scalar. Stacking those scalars gives the layer's output vector, which becomes the input to the next layer.

If a layer has n_{out} neurons and receives n_{in} inputs, then every neuron has its own n_{in} weights and its own bias. Stacking the weight vectors as rows gives one matrix, so the whole layer computes in a single step:

$$z = Wa_{\text{prev}} + b, \qquad a = f(z)$$

with W of shape $(n_{\text{out}} \times n_{\text{in}})$, and b, z, a of length n_{out}. The activation f is applied to each coordinate independently.

Example: a $2 \to 2 \to 1$ network.

- **Input layer:** just x, the raw inputs. It has no weights and does no computation.
- **Hidden layer:** 2 neurons, each seeing both inputs ($W^{(1)}$ is 2×2). It is called hidden because its outputs are neither the inputs nor the final prediction.
- **Output layer:** 1 neuron, seeing both hidden outputs ($W^{(2)}$ is 1×2), producing $\hat{y}$.

A layer where every neuron sees every input from the previous layer is called a **fully connected** (or dense) layer. A network with several of these in a row is an MLP.

Do Input Layer Neurons Perform Computation?

No. The input layer's "neurons" are just placeholders that hold the raw input values $(x_1, x_2, \dots)$. They have no weights, no bias and no activation, so nothing is computed there.

The first real computation happens in the first hidden layer, which takes the input values and applies $f(Wx+b)$. The weight matrix $W^{(1)}$ is described as connecting the input layer to the first hidden layer, but it belongs to the hidden layer's neurons: its shape is $(n_{\text{hidden}} \times n_{\text{input}})$.

This is also why a $2 \to 2 \to 1$ network is usually called a "2-layer network" (hidden + output). The input layer is drawn as circles for visualization, but it is not counted because it does no computation.

Input Layer Size for a Tabular Dataset

Suppose a tabular dataset has 1000 rows and 4 columns. The number of input neurons depends on what the columns are.

- **All 4 columns are input features (target is separate):** 4 input neurons, one per feature. The number of rows does not matter; each row is passed in as a vector $x \in \mathbb{R}^4$, and the 1000 rows are 1000 training examples.
- **One of the 4 columns is the target (label):** only 3 are features, so the input layer has 3 neurons. The target column is what the output layer is compared against in the loss, not something fed in.
- **Preprocessing can change the input size:** a categorical column with k categories, one-hot encoded, becomes k input neurons instead of 1. Dropping a column, or adding engineered features, changes the count too.

The rule is: **input layer size = number of numeric input values per row after preprocessing**, not the raw column count. The label is excluded.

Since there are 1000 rows, they are typically passed in batches (say 32 at a time). The input is then a matrix of shape $(\text{batch} \times n_{\text{features}})$, but the network's structure is unchanged: batch size never affects the number of neurons or weights.

Output Layer Design Based on the Label Type

For a dataset with 3 numerical features and 1 label, the input layer has 3 neurons. The label is not fed in; it only appears in the loss, where it is compared against the network's output.

The output layer depends on the label type:

- **Regression (continuous label):** 1 output neuron, usually with no activation (or identity).
- **Binary classification:** 1 output neuron with sigmoid.
- **Multi-class classification with k classes:** k output neurons with softmax.

A network such as $3 \to 8 \to 4 \to 1$ would work for regression or binary classification. The hidden sizes (8 and 4) are a free choice; only the input size (3) and the output size (set by the task) are fixed by the data.

One-Hot Encoding of Categorical Features

A neuron computes $w^\top x + b$, so every input has to be a number, and the number's size has to mean something. A categorical column like color $= \{\text{red}, \text{green}, \text{blue}\}$ is just labels. If we encode them as red $= 1$, green $= 2$, blue $= 3$, we tell the network that blue is "three times red" and green sits "between" them. The weight multiplies that number, so the network is forced to treat the categories as points on a line with a fixed order and spacing, which is usually false.

One-hot encoding avoids this by giving each category its own 0/1 column:

color	is_red	is_green	is_blue
red	1	0	0
green	0	1	0
blue	0	0	1

One original column becomes k columns (here $k = 3$), and exactly one of them is 1 in each row. Since the input layer has one neuron per input value, those k columns need k input neurons.

This also fixes the weight problem. The first hidden neuron now has a separate weight for each category: $w_{\text{red}}, w_{\text{green}}, w_{\text{blue}}$. Its pre-activation is

$$z = w_{\text{red}} \cdot \text{is_red} + w_{\text{green}} \cdot \text{is_green} + w_{\text{blue}} \cdot \text{is_blue} + (\text{other terms}) + b$$

For a red row, only w_{red} contributes. For a green row, only w_{green} does. So the network learns an independent effect for each category, with no imposed ordering or spacing.

Example: a dataset has 3 numerical features plus one categorical column with 4 categories. After one-hot encoding, each row is a vector of length $3 + 4 = 7$, so the input layer has 7 neurons.

Notes:

- If a column has thousands of categories (like a zip code), one-hot becomes very wide. In that case, **embeddings** are used, which map each category to a short learned vector instead.
- If the categories have a natural order (small < medium < large), encoding them as 1, 2, 3 can be reasonable, because the ordering is real.

Why Each Feature Gets One Input Slot Instead of Feeding All Features into One Neuron

The input layer is not made of computing neurons. It is just the input vector, with one slot per value, so "one neuron per feature" simply means "one slot per number." A neuron can take multiple inputs, but that happens in the next layer, not in the input layer.

Feeding all features into one neuron is possible, but that neuron is a hidden-layer neuron, and it has two consequences:

1. **It compresses the data.** With 3 features, one neuron computes $z = w_1x_1 + w_2x_2 + w_3x_3 + b$ and outputs a single scalar. Everything the next layer sees is that one number, so many different feature combinations collapse to the same value and information is lost. For example, with $w = (1, 1, 1)$, the rows $(1, 2, 3)$ and $(3, 2, 1)$ are indistinguishable.
2. **It limits what the network can learn.** The network can only use one weighted combination of the features. A layer with several neurons learns several different combinations (each with its own weight vector), and later layers can combine those into more complex patterns. One neuron gives only a single "view" of the data.

So the structure is:

- **Input layer:** holds the raw values, 3 slots, no computation, nothing lost.
- **First hidden layer:** every neuron takes all 3 features as input, and the number of neurons (a free choice) sets how many combinations are learned.

The input layer must keep the features separate so that the hidden neurons can each apply their own weights to each individual feature.

Weight Matrix Shapes: Example $4 \to 4 \to 3 \to 1$

Consider a dataset with 1000 rows and 5 columns (4 numeric features, 1 label), and a network with architecture $4 \to 4 \to 3 \to 1$.

Hidden layer 1. The weight matrix has shape (4×4) using the convention W of shape $(n_{\text{out}} \times n_{\text{in}})$. The layer has 4 neurons ($n_{\text{out}} = 4$) and receives the 4 input features ($n_{\text{in}} = 4$). Each row of W holds one neuron's 4 weights, and the bias b has length 4. The layer computes $z = Wx + b$, with W (4×4) times x (4×1) giving z (4×1).

Parameters for the whole architecture:

- Hidden layer 1: $W^{(1)}$ is 4×4, $b^{(1)}$ has length $4 \to 16 + 4 = 20$ parameters
- Hidden layer 2: $W^{(2)}$ is 3×4, $b^{(2)}$ has length $3 \to 12 + 3 = 15$ parameters
- Output layer: $W^{(3)}$ is 1×3, $b^{(3)}$ has length $1 \to 3 + 1 = 4$ parameters

Total: 39 parameters.

Weight Matrix Shape Convention: $(n_{\text{out}} \times n_{\text{in}})$ vs. $(n_{\text{in}} \times n_{\text{out}})$

Why is $W^{(2)}$ written as 3×4 and not 4×3? It depends on the convention. Here we use $(n_{\text{out}} \times n_{\text{in}})$, where each row holds one neuron's weights.

Hidden layer 2 has 3 neurons and receives 4 inputs:

- Each neuron needs 4 weights (one per input), so each neuron is a row of length 4.
- There are 3 neurons, so there are 3 rows.
- $W^{(2)}$ is therefore 3×4.

This makes the math work with the column-vector form $z = Wa + b$: (3×4) times (4×1) gives (3×1), a vector with one value per neuron.

The alternative 4×3 is the other common convention, $(n_{\text{in}} \times n_{\text{out}})$, where each *column* holds one neuron's weights. It is valid, but the computation is then written as $z = a^\top W + b$, i.e. (1×4) times (4×3) gives (1×3). PyTorch's `nn.Linear` stores the weight as $(n_{\text{out}} \times n_{\text{in}})$, while Keras's `Dense` stores it as $(n_{\text{in}} \times n_{\text{out}})$. The two are transposes of each other, with the same 12 numbers.

Whichever convention is chosen, it must be used consistently so that the shapes multiply correctly. In general, a layer's weight shape is (neurons in this layer $\times$ neurons in the previous layer) under the first convention.

Neurons and Logistic Regression

The big picture: **one neuron = a weighted score + a squashing step**. The affine score is introduced first, logistic regression shows it is a one-neuron network, two worked examples use real numbers, and finally we derive why one formula turns out so clean.

The Neuron as an Affine Score

Plain idea: A neuron takes some numbers in, mixes them using "importance" values (weights), adds a nudge (bias), then passes the result through a bending function.

Think of grading a student: score = $(2 \times \text{exam}) + (1 \times \text{homework}) + 5$ bonus points. That is the first half. The second half might be "cap the final grade between 0 and 1."

Equation: Neuron Score and Activation

$$z = w^\top x + b, \qquad a = f(z)$$

Symbols:

- x = the inputs, a list of d numbers (e.g., exam mark, homework mark)

- w = the weights, a list of d numbers, one per input. Bigger weight means that input matters more
- $w^\top x$ = multiply each input by its weight, then add everything up
- b = bias, a single number added at the end (a shift)
- z = the raw score (called the pre-activation)
- f = the activation function, a rule that reshapes z
- a = the neuron's final output

What it says: First compute a straight-line score z. Then pass z through f to get the output a.

Conditions: Always holds. "Affine" just means "straight-line style": plotted against any single input, z is a straight line.

Special values:

- $w = 0$: the neuron ignores the input, and z is just b.
- $b = 0$: the line passes through the origin, so zero input gives zero score.
- f = identity ($f(z) = z$): the neuron stays purely straight-line, no curves at all.

Equation: Stacking Affine Layers Collapses

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$$

Symbols:

- W_1, W_2 = weight tables (matrices) for layer 1 and layer 2
- b_1, b_2 = the biases of each layer
- x = the input

What it says: If you stack two straight-line layers with no bending function between them, you could replace both with **one** straight-line layer. The left side is "do layer 1, then layer 2." The right side is the same thing as a single layer with weights W_2W_1 and bias $W_2b_1 + b_2$. (You get it by multiplying out the brackets.)

Why it matters: Without a bending function between layers, a 100-layer network is no more powerful than a 1-layer one. The activation function f is what makes depth useful.

Conditions: Holds whenever the matrix sizes fit together so the multiplication is possible.

Special values: If $W_1 = 0$, everything collapses to just b_2, so the input is ignored. If $W_2 = 0$, same result.

Logistic Regression as a One-Neuron Network

Plain idea: You want a yes/no answer (spam or not, $y = 1$ or 0). A neuron gives you any number, but you want a probability between 0 and 1. The sigmoid squeezes any number into that range. Then you need a way to measure how wrong the prediction was.

Equation: Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Symbols:

- σ (sigma) = the sigmoid function

- z = the raw score from the neuron
- e = a special constant ≈ 2.718
- e^{-z} = e raised to the power minus z

What it says: It takes any number and squashes it into the range between 0 and 1, which can be read as "probability of yes."

Conditions: Works for every real number z.

Special values:

- $z = 0$: $e^0 = 1$, so $\sigma = 1/2$. Total uncertainty.
- $z \to$ very large positive: $e^{-z} \to 0$, so $\sigma \to 1$ (confident "yes").
- $z \to$ very large negative: $e^{-z} \to$ huge, so $\sigma \to 0$ (confident "no").
- It never actually reaches 0 or 1, which is why the range is written as $(0, 1)$.

Equation: Prediction

$$\hat{y} = \sigma(w^\top x + b)$$

Symbols: $\hat{y}$ (y-hat) is the predicted probability. The rest are as before.

What it says: Compute the straight-line score, then squash it. That is the whole prediction.

Equation: Binary Cross-Entropy Loss

$$L = -\Big[\, y \log \hat{y} + (1 - y) \log(1 - \hat{y}) \,\Big]$$

This is the "how wrong was I?" number.

Symbols:

- L = loss (the penalty; lower is better)
- y = the true answer (0 or 1)
- $\hat{y}$ = your predicted probability
- log = natural logarithm

What it says: Because y is either 0 or 1, one half always disappears:

- If $y = 1$: $L = -\log \hat{y}$. Penalty is small when $\hat{y}$ is near 1, big when $\hat{y}$ is near 0.
- If $y = 0$: $L = -\log(1 - \hat{y})$. Penalty is small when $\hat{y}$ is near 0, big when $\hat{y}$ is near 1.

So it punishes you for being confident and wrong.

Conditions: Needs $y \in \{0, 1\}$ and $\hat{y}$ strictly between 0 and 1.

Special values:

- $\hat{y} = 1/2$: $L = \log 2 \approx 0.693$ either way (a coin-flip guess).
- $y = 1$ and $\hat{y} \to 0$: $\log \hat{y} \to -\infty$, so $L \to \infty$. Infinite penalty for being completely sure and wrong.
- $\hat{y}$ exactly 0 or 1: $\log(0)$ is undefined, which is why code "clips" $\hat{y}$ slightly away from the edges.

Two Key Facts (Previewed)

$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big), \qquad \frac{\partial L}{\partial z} = \hat{y} - y$$

Symbols:

- $\sigma'(z)$ = the slope of the sigmoid at z (how fast it changes)
- $\partial L/\partial z$ = how much the loss changes if you nudge z a tiny bit

What they say: The sigmoid's slope can be written using its own output. And the loss's sensitivity to z is just "prediction minus truth," which is very simple. The derivation below proves the second one.

Worked Example: Logistic Forward Pass

Setup: $w = 2$, $b = -2$, $x = 1$, $y = 1$ (true answer: yes).

Step by step:

- $z = wx + b = 2 \cdot 1 - 2 = \mathbf{0}$
- $\hat{y} = \sigma(0) = \mathbf{1/2}$
- $L = -\log(1/2) = \ln \mathbf{2} \approx \mathbf{0.693}$

(The loss formula reduces to $-\log \hat{y}$ because $y = 1$, so the second half vanishes.)

Reading the number: $\hat{y} = 1/2$ means the neuron is basically guessing, like flipping a coin. A loss of $\ln 2 \approx 0.693$ is the "coin flip loss." If your classes are balanced (half yes, half no) and you start with random weights, your starting loss should be near 0.693. If it starts at, say, 50, something is likely wrong.

Worked Example: Logistic Backward Pass

Plain idea: "Backward" means asking: *which way should I nudge w and b to reduce the loss?* The answer is the slope (gradient) of the loss with respect to each.

Step by step (same numbers: $\hat{y} = 1/2$, $y = 1$, $x = 1$):

1. $\dfrac{\partial L}{\partial z} = \hat{y} - y = \dfrac{1}{2} - 1 = -\dfrac{1}{2}$. The negative sign means: *increasing z would lower the loss.*
2. $\dfrac{\partial L}{\partial w} = \dfrac{\partial L}{\partial z} \cdot x = -\dfrac{1}{2} \cdot 1 = -\dfrac{1}{2}$. Since $z = wx + b$, a nudge to w changes z by x times as much, so you multiply by x. This is the chain rule: multiply the slopes along the path.
3. $\dfrac{\partial L}{\partial b} = \dfrac{\partial L}{\partial z} = -\dfrac{1}{2}$. A nudge to b changes z by exactly that amount (the factor is 1).

Equation: Gradient Descent Update

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

Symbols:

- $\leftarrow$ = "replace with the new value"
- η (eta) = learning rate, the step size (a small positive number you choose)
- $\partial L/\partial w$ = the slope computed above

What it says: Move w a small step *opposite* to the slope. Here: $w \leftarrow 2 - \eta(-1/2) = \mathbf{2} + \eta/\mathbf{2}$.

Result: w grows, so z on this input becomes positive, so $\hat{y}$ moves up toward 1 (the correct answer).

Conditions: η must be positive. If it is too big you overshoot; too small and learning is slow.

Special values:

- $\eta = 0$: nothing changes, no learning.
- Gradient $= 0$: no movement, which happens at a minimum, a flat spot, or a "dead" path.

Derivation of $\partial L/\partial z = \hat{y} - y$

Plain idea: The loss depends on $\hat{y}$, and $\hat{y}$ depends on z. So the chain is $z \to \hat{y} \to L$. To find how L changes with z, find the slope of each link and multiply them (chain rule):

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z}$$

Setup

- $L = -\big[\, y \log \hat{y} + (1-y) \log(1-\hat{y}) \,\big]$ is the loss
- $\hat{y} = \sigma(z)$ is the prediction
- $\sigma(z) = \dfrac{1}{1+e^{-z}}$ is the sigmoid
- y is the true label (0 or 1), a fixed number, not something that changes with z

Step 1: Slope of the Sigmoid, $\partial \hat{y}/\partial z$

Write $\sigma(z) = (1+e^{-z})^{-1}$. Differentiate:

$$\sigma'(z) = (1+e^{-z})^{-2} \cdot e^{-z}$$

Split this into two fractions:

$$\sigma'(z) = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}}$$

The first fraction is $\sigma(z) = \hat{y}$. For the second, note that $1 - \sigma(z) = \dfrac{e^{-z}}{1+e^{-z}}$, so it equals $1-\hat{y}$. Therefore:

$$\frac{\partial \hat{y}}{\partial z} = \hat{y}(1-\hat{y})$$

Step 2: Slope of the Loss with Respect to $\hat{y}$, $\partial L/\partial \hat{y}$

Differentiate each term (the derivative of $\log u$ is $1/u$, and the derivative of $\log(1-\hat{y})$ picks up a minus sign):

$$\frac{\partial L}{\partial \hat{y}} = -\left[\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}}\right] = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}$$

Put both fractions over the common denominator $\hat{y}(1-\hat{y})$:

$$= \frac{-y(1-\hat{y}) + (1-y)\hat{y}}{\hat{y}(1-\hat{y})}$$

Expand the top: $-y + y\hat{y} + \hat{y} - y\hat{y} = \hat{y} - y$ (the $y\hat{y}$ terms cancel). So:

$$\frac{\partial L}{\partial \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1-\hat{y})}$$

Step 3: Multiply

$$\frac{\partial L}{\partial z} = \frac{\hat{y} - y}{\hat{y}(1-\hat{y})} \cdot \hat{y}(1-\hat{y})$$

The $\hat{y}(1-\hat{y})$ on top and bottom cancel:

$$\frac{\partial L}{\partial z} = \hat{y} - y$$

Sanity Check

With the worked example's numbers, $\hat{y} = 1/2$ and $y = 1$ give $\hat{y} - y = -1/2$, which matches the backward pass above.

Conditions

This holds only for **sigmoid output + binary cross-entropy** together, with $0 < \hat{y} < 1$ (otherwise Step 2 divides by zero). With a different loss or activation, the factors would not cancel.

Special Values

- $\hat{y} = y$: gradient is 0, so there is nothing to fix.
- $y = 1$, $\hat{y} \to 0$ (confident and wrong): gradient $\to -1$, a strong push. It does not vanish, even though $\sigma' \to 0$ and $\partial L / \partial \hat{y} \to \infty$ individually. The two extremes cancel exactly.

Why Squared Error Is a Poor Match for Sigmoid

If you used $L = \frac{1}{2}(\hat{y} - y)^2$ instead, the same chain gives:

$$\frac{\partial L}{\partial z} = (\hat{y} - y)\, \sigma'(z)$$

Nothing cancels, so there is a leftover $\sigma'(z)$.

Why that is a problem: When the neuron is very confident (z very large or very small), $\sigma'(z) \approx 0$. So even if the neuron is **confidently wrong**, the gradient is nearly zero and the weights barely move. Learning stalls exactly when it is needed most. Binary cross-entropy cancels that σ' factor, which is why sigmoid + BCE is the standard pairing for yes/no problems.

Summary

A neuron = straight-line score + squash; sigmoid turns scores into probabilities; cross-entropy punishes confident mistakes; and the gradient of that pair is simply *prediction* − *truth*, which tells you which direction to nudge the weights.